AI Food Nutrition App – Full Product Requirement Document

This document contains: 1. MVP (Minimum Viable Product) requirements 2. Full product vision 3. Backend requirements 4. Frontend requirements 5. Database design 6. API design 7. AI integration flow 8. Deployment strategy 9. Security requirements 10. Step-by-step development plan The document is written so it can be directly used with AI coding tools like GitHub Copilot, Cursor AI, Claude, or ChatGPT for generating code.

1. Product Vision

Build an AI-powered nutrition tracking application that allows users to:
- Upload food photos
- Detect food items automatically
- Estimate calories and nutrients
- Track health and fitness goals
- Receive AI-powered recommendations

Target users:
- Gym users
- Weight loss users
- Health-conscious users
- Athletes
- Diet-focused users

2. MVP Scope (Build in 2–3 Weeks)

The MVP should ONLY include:

Core Features:
- User registration/login
- Upload food image
- AI food detection
- Nutrition calculation
- Meal history
- Daily calorie tracking
- Protein/fat/carb tracking
- Simple dashboard

DO NOT include:
- Social features
- Subscription system
- Meal planning
- AI chatbot
- Wearable integration
- Video processing
- Custom ML training

3. Recommended Tech Stack

Frontend:
- Next.js
- Tailwind CSS
- TypeScript

Backend:
- Java 21
- Spring Boot
- Spring Security
- JWT Authentication
- Maven

Database:
- PostgreSQL

Storage:
- Supabase Storage

Hosting:
- Vercel (frontend)
- Render/Railway (backend)

AI APIs:
- OpenAI Vision API

Nutrition APIs:
- USDA FoodData Central API

4. Frontend Requirements

Pages Required:

1. Login Page
- Email/password login
- JWT storage
- Redirect after login

2. Signup Page
- Create account
- Form validation

3. Dashboard
- Total calories today
- Protein progress
- Fat progress
- Carb progress
- Upload meal button

4. Upload Meal Page
- Upload image
- Preview image
- Submit for analysis

5. Results Page
- Detected foods
- Calories
- Protein
- Fat
- Carbs
- Cholesterol
- Confirm/save button

6. Meal History Page
- Previous meals
- Daily totals
- Nutrition summaries

7. Profile Page
- Weight
- Height
- Age
- Goal
- Activity level

5. Backend Requirements

Modules:

1. Authentication Module
- Signup
- Login
- JWT generation
- Password encryption

2. User Module
- Profile management
- Update goals
- Update weight

3. Meal Module
- Save meal
- Get meal history
- Delete meal

4. AI Analysis Module
- Send image to OpenAI Vision
- Parse response
- Extract foods and portions

5. Nutrition Module
- Fetch nutrition values
- Calculate totals

- Store macros

6. Analytics Module
- Daily totals
- Weekly summaries
- Goal progress

6. Database Design

Tables:

Users
- id
- name
- email
- password_hash
- age
- weight
- height
- fitness_goal
- activity_level
- created_at

Meals
- id
- user_id
- image_url
- meal_time
- created_at

Meal_Items
- id
- meal_id
- food_name
- calories
- protein
- fat
- carbs
- cholesterol
- serving_size

Daily_Summaries
- id
- user_id
- total_calories
- total_protein
- total_fat
- total_carbs
- summary_date

7. API Requirements

Authentication APIs:
POST /api/auth/signup
POST /api/auth/login

User APIs:
GET /api/users/profile
PUT /api/users/profile

Meal APIs:
POST /api/meals/upload
GET /api/meals/history
DELETE /api/meals/{id}

Analytics APIs:
GET /api/analytics/daily
GET /api/analytics/weekly

8. AI Food Detection Flow

Flow:
1. User uploads image
2. Frontend sends image to backend
3. Backend uploads image to storage
4. Backend sends image URL to OpenAI Vision API
5. AI returns:
- food names
- estimated portions
6. Backend queries nutrition API
7. Backend calculates totals
8. Backend stores meal
9. Frontend displays results

9. OpenAI Prompt Strategy

Use structured prompts.

Example Prompt:
'Analyze this food image.
Identify all visible foods.
Estimate serving size.
Return JSON only.

Required fields:
- food_name
- serving_size
- calories
- protein
- fat
- carbs'

Use JSON response formatting for easier parsing.

10. Security Requirements

Must Implement:
- HTTPS
- Password hashing using BCrypt
- JWT authentication
- Input validation
- Rate limiting
- File type validation
- Secure storage

Avoid:
- Public image URLs
- Hardcoded secrets
- Open database access

11. Folder Structure Recommendation

Backend Structure:

src/main/java/com/app
/auth
/users
/meals
/nutrition
/analytics
/config
/security
/dto
/entity
/repository
/service
/controller

Frontend Structure:

/app
/components
/pages
/services
/hooks
/utils
/types

12. MVP Development Plan

Week 1:
- Setup backend
- Setup database
- JWT auth
- Build frontend pages

Week 2:
- Upload flow
- AI integration
- Nutrition calculation
- Save meals

Week 3:
- Analytics
- Testing
- Deployment
- Beta launch

13. Full Product Vision

Future Features:

- AI meal recommendations
- Voice meal logging
- Barcode scanner
- Wearable integration
- AI nutrition coach
- Weekly diet plans
- Indian food optimization
- Subscription system
- Fitness trainer dashboard
- Family accounts
- Restaurant nutrition scanner

14. Full Product Architecture

Frontend:
- React Native mobile apps
- Web dashboard

Backend:
- Spring Boot microservices

Services:
- Auth service
- Meal service
- AI service
- Recommendation engine
- Analytics engine

Infrastructure:
- Docker
- Kubernetes
- Redis caching
- CDN
- Monitoring

15. Recommendation Engine Logic

Examples:

IF:
- protein intake low
- user goal = muscle gain

THEN:
Suggest:
- eggs
- paneer
- chicken
- whey
- dal

IF:
- calories exceed target

THEN:
Suggest lighter dinner options.

16. Monetization Strategy

Free Plan:
- Limited scans/day
- Basic analytics

Premium:
- Unlimited scans
- AI coach
- Advanced analytics
- Diet plans
- Fitness integrations

Potential pricing:
■199–499/month

17. Startup Strategy Recommendations

Do:
- Launch fast
- Get real users
- Improve based on feedback
- Focus on retention

Avoid:
- Overengineering
- Building too many features
- Training custom ML models initially
- Spending large money before validation

18. Patent & Legal Recommendation

Do NOT focus on patents initially.

Why:
- Similar apps already exist
- Execution matters more

Instead:
- Build strong brand
- Protect user data
- Trademark app name later
- Focus on speed and user experience

19. AI Coding Instructions

Use GitHub Copilot/Cursor prompts like:

'Generate Spring Boot REST API for meal upload with JWT auth.'

'Generate Next.js nutrition dashboard with Tailwind.'

'Generate PostgreSQL schema for nutrition tracking app.'

'Generate OpenAI Vision integration service in Spring Boot.'

AI coding tools should generate:
- CRUD APIs
- DTOs
- Database models
- Frontend components
- Validation
- Authentication

20. Final Recommendation

The fastest successful path is:

1. Build web MVP first
2. Keep backend monolithic
3. Use OpenAI APIs instead of training models
4. Launch in 21 days
5. Get 20–100 users
6. Improve based on real feedback

Success depends more on:
- Consistency
- Simplicity
- Retention
than advanced AI initially.